

**2022 Controller Revision: Proof of Function****11/28/2022****Rev: 0A****ABSTRACT:**

This document serves two purposes; primarily, it acts as a record of all testing carried out on the first controller developed for the 2022 MUPP control system overhaul. Secondly, this document also acts as a "road map", for any persons conducting control system updates to the MUPP in the future. Included within this document, is the testing of all four functions of the designed controller, these being single-loop, cascade-loop, cascade feedback augmented feedforward, and outer loop over OPC control schemes. The methodology of testing is described as well as the results, and some discussion regarding the interpretation of results. Following testing, it was concluded that the developed controller acted as intended, and deployment of the remaining controllers may proceed.

Contents

List of Tables	3
List of Figures	4
Document Control and Revision History.....	5
Definitions.....	6
1.0 Purpose	7
2.0 Scope.....	8
3.0 Methodology.....	9
3.1 Cascade Vs. Single-loop Testing.....	11
3.2 Feedforward Control Testing	11
3.3 OPC Control Testing	12
4.0 Results.....	16
5.0 Discussion.....	21
5.1 Cascade and Single-Loop Testing.....	21
5.2 Feedforward Testing	21
5.3 OPC Performance.....	21
6.0 Conclusion.....	22

List of Tables

Table 1: Document Control and Revision History 5

Table 2: A complete list of manipulated variables in the Murdoch University Pilot Plant. 7

List of Figures

Figure 1: MUPP OPC program created for retrieving data from the PP OPC server.	10
Figure 2: Block diagram of the designed controller and simplified physical system: Control flow can be configured from EPKS HMI such that MATLAB achieves outer-loop cascade control via OPC (a) OR through EPKS CM (b). Outer-loop controller output is then fed into the inner-loop controller (c) housed in the EPKS CM, the output of which is fed into the physical system (d). <i>NOTE: Outer-loop controllers can also be fed directly into the physical system, thus becoming the controller under a single-loop control scheme.</i>	13
Figure 3: MUPP OPC program created for testing MUPP control from MATLAB via OPC.....	15
Figure 4: A comparison of Cascade versus single-loop feedback control, plots show BMT liquid level (<i>top</i>) and controlled feed pump speed (<i>bottom</i>). The process dynamics observed are in response to a change in setpoint from 50 to 60% maximum liquid level.....	16
Figure 5: A comparison of Cascade (<i>top</i>) versus single-loop feedback (<i>bottom</i>) control plots show inflow into BMT compared to the outer-loop manipulated variable. The process dynamics observed are in response to a change in setpoint from 50 to 52% maximum liquid level.	17
Figure 6: A comparison of Cascade (<i>top</i>) versus single-loop feedback (<i>bottom</i>) control plots show inflow into BMT compared to the outer-loop manipulated variable. The process dynamics observed are in response to a change in setpoint from 52 to 50% maximum liquid level.	17
Figure 7: Response to step testing of FP and CRP, and the corresponding liquid levels (<i>top</i>) and flow rates into BMT (<i>bottom</i>) observed as a result. All step testing was implemented as a 2% increase in pump speed, then normalized based on the associated flow rate.	18
Figure 8: A comparison of Cascade Feedback Augmented Feedforward versus, Cascade-only feedback control disturbance rejection qualities. The process dynamics observed are a response to a 20% step in CRP speed from 30-50%, resulting in a ~8% increase in flow rate % from ~16-24% disturbance flow rate. (<i>top</i>) shows liquid level as a percentage of maximum height in the BMT, (<i>bottom</i>) shows disturbance variable (FT-347) for each controller during testing.....	18
Figure 9: Performance of cascade controllers to a 5% increase in SP from 50 to 55%. The comparison shows the outer loop housed in the EPKS control module versus the outer loop housed in the MATLAB program and communicated via OPC to the EPKS control module. In both cases, configuration parameters are identical, (<i>top</i>) shows the liquid level in BMT, and (<i>bottom</i>) shows pump speed.	19
Figure 10: Performance of cascade controllers to a 5% increase in SP from 55 to 50%. The comparison shows the outer loop housed in the EPKS control module versus the outer loop housed in the MATLAB program and communicated via OPC to the EPKS control module. In both cases, configuration parameters are identical, (<i>top</i>) shows the liquid level in BMT, and (<i>bottom</i>) shows pump speed.	20

Document Control and Revision History

Table 1: Document Control and Revision History

Revision	Date	Author(s)	Reviewed by	Comments/Details
0A	5/10/22	Nicholas O'Keeffe		Working Document

Definitions

BMT	Ball Mill Tank
CFBFF	Cascade Feedback augmented Feedforward controller
CRP	Cyclone Recycle Pump
CSTR	Continuously Stirred Tank Reactor
CUFP	Cyclone Underflow Pump
EPKS	Experion Process Knowledge System
FCV	Flow Control Valve
FDP	Flow Disturbance Pump
FP	Feed Pump
FT	Flow Transmitter
LUFP	Lamella Underflow Pump
MUPP	Murdoch University Pilot Plant
MV	Manipulated variable
NUFP	Needle Tank Underflow Pump
OPC	Open Platform Communications
PP	Product Pump
PV	Process variable
SP	Setpoint
TF	Transfer Function

1.0 Purpose

The primary purpose of this document is to record all testing of the controller developed for the MUPP control system overhaul. Included within the plant overhaul is the rollout of controllers for each manipulated variable (MV) defined in the MUPP (*Table 2*). The testing took place using the first controller completed, that of the feed pump (*FP-141*). The controller was developed in such a way as to allow it to be relatively easily deployed to all other MUPP MVs once its construction was completed. Thus, in testing the first MVs controller, the goal is to prove its various components' function before it is rolled out to the remaining MVs.

This document also serves a secondary purpose, that of a "*road map*", for any persons conducting control system updates to the MUPP in the future. The hope is that, with the aid of this document, a scheme for EPKS controller testing can be inferred and developed so that any future controller build can be adequately tested.

Table 2: A complete list of manipulated variables in the Murdoch University Pilot Plant.

Asset:	Tag Number:	Asset:	Tag Number:
Feed Pump	FP-141	Non-Linear Tank input FCV	FCV-541
Ball Mill Pump	BMP-241	CSTR-1 input FCV	FCV-570
Cyclone Recycle Pump	CRP-341	NUFP to BMT FCV	FCV-574
Cyclone Underflow Pump	CUFP-361	CSTR-1 steam addition valve	FCV-622
Lamella Underflow Pump	LUFP-421	CSTR-2 steam addition valve	FCV-642
Flow Disturbance Pump	FDP-521	CSTR-3 steam addition valve	FCV-662
Needle Tank Underflow Pump	NUFP-561	Plant outflow FCV	FCV-688
Product Pump	PP-681	CSTR-3 reflux FCV	FCV-690

2.0 Scope

This document includes information ascertaining all aspects of testing the first controller developed as part of the 2022 MUPP control system overhaul. The controller being tested is that of the Feed Pump (*FP-141*); the controller is designed such that it can be easily configured from the Experion Station HMI to function in one of several configurations, these being:

1. As a single-loop feedback controller,
2. As a Cascade feedback controller,
3. As a feedback-augmented feedforward controller,
4. As a cascade feedback-augmented feedforward controller,

In addition to these conventional configurations, it can be re-routed such that the outer-loop controller of a cascade feedback controller or the entire controller is housed in a third-party application. In these cases, data acquisition from and writing to the EPKS server is handled via OPC. This document describes the methodology, results, and interpretation of the results of testing of all the aforementioned features.

3.0 Methodology

The developed EPKS Control Module (CM) based controller is made up of four interacting elements:

1. An outer-loop PID controller.
2. An inner-loop PID controller.
3. A feedforward controller with output injected into the loop immediately after the outer loop controller.
4. Configurable routing, such that, when activated, the outer-loop controller and feedforward arm can be replaced by a controller on a third-party application and routed into the EPKS CM via OPC.

Dependent on the configuration of the CM from the Experion Station HMI, the controller can operate as:

- A single-loop feedback controller,
- A cascade feedback controller,
- A feedback-augmented feedforward (FBFF) controller,
- A cascade feedback augmented feedforward (CFBFF) controller,

On top of the cases mentioned above, when OPC control is active, the outer loop and feedback arms are deactivated, and externally configured controllers can be fed either into the inner-loop controller or directly into the physical system. As a result, any control schemes outside the scope of the EPKS are also made available.

As each of the above cases needed to be tested to show the function of the developed CM, a method of data collection and analysis was required. Data collection and analysis were achieved using a MATLAB program to connect to the MUPPs OPC server and collect prescribed data points. Figure 1 shows the commented program; however, simply speaking, the program's function can be broken down into four broad steps (*For further clarification of program function, consult Figure 1*):

1. Initializing communication with the MUPP OPC server.
2. Definition of variables to be read from the server.
3. Initialization of storage variables (i.e., the first read of data from the OPC server and the initialization of variables to which this read data is saved).
4. The continuous looping of data reads at a rate of one sample per second.

With the program for data collection developed, controller testing was achieved by running various tests (*which will be elaborated upon in the proceeding sections*) before exporting the data relevant to each test as a MATLAB structure. Once the structures were exported, separate MATLAB programs could import this data, where analysis and/or plotting could be performed.

```

% MUPP -- OPC Data Collection
clear, close all

%%      Initialization of Server Comms
% Connect to MUPP OPC Server
server = opcda('ppserver1', 'HWHsc.OPCServer');
connect(server);

% Establish OPC Group i.e. EPKS Control Module to retrieve data from
grp = addgroup(server, 'FP_REF_141-R1A');

% Define variables to be read
fp141_read = additem(grp, 'FP_REF_141-R1A.FP_141.PV', 'double');
pv1_read = additem(grp, 'FP_REF_141-R1A.PV1.PV', 'double');
pv2_read = additem(grp, 'FP_REF_141-R1A.PV2.PV', 'double');
dv_read = additem(grp, 'FP_REF_141-R1A.DV.PV', 'double');
sp_read = additem(grp, 'FP_REF_141-R1A.SP.PV', 'double');
mv_read = additem(grp, 'FP_REF_141-R1A.MV_OuterLoop.PV', 'double');

%%      Initialization of read variables
% Perform first read >> convert read to cell
tempreadfp = struct2cell(read(fp141_read));
tempreadpv1 = struct2cell(read(pv1_read));
tempreadpv2 = struct2cell(read(pv2_read));
tempreaddv = struct2cell(read(dv_read));
tempreadsp = struct2cell(read(sp_read));
tempreadmv = struct2cell(read(mv_read));

% Take second entry from initial read (This is the value of the
% variable) and convert to double.
cont1.fp141(1) = cell2mat(tempreadfp(2)); % \
cont1.pv1(1) = cell2mat(tempreadpv1(2)); % | Save variables as structure
cont1.pv2(1) = cell2mat(tempreadpv2(2)); % | so they can be easily
cont1.dv(1) = cell2mat(tempreaddv(2)); % | saved and retrieved by
cont1.sp(1) = cell2mat(tempreadsp(2)); % | other programs.
cont1.mv(1) = cell2mat(tempreadmv(2)); % /

%%      Continuous OPC read loop start
n = 1;
cont1.t(1) = 0;
while true
    pause(1)

% UNCOMMENT FOR REAL TIME PLOTTING
% subplot(2,1,1)
% plot(cont1.t, cont1.pv1, 'b', cont1.t, cont1.sp, 'r')
% grid on
% subplot(2,1,2)
% plot(cont1.t, cont1.mv, 'b', cont1.t, cont1.pv2, 'r')
% grid on
% hold on

% Progress n and t
n = n+1; cont1.t(n) = (n-1);

% Perform next read >> convert read to cell
tempreadfp = struct2cell(read(fp141_read));
tempreadpv1 = struct2cell(read(pv1_read));
tempreadpv2 = struct2cell(read(pv2_read));
tempreaddv = struct2cell(read(dv_read));
tempreadsp = struct2cell(read(sp_read));
tempreadmv = struct2cell(read(mv_read));

% Take second entry from initial read (This is the value of
% the variable) and convert to double.
cont1.fp141(n) = cell2mat(tempreadfp(2));
cont1.pv1(n) = cell2mat(tempreadpv1(2));
cont1.pv2(n) = cell2mat(tempreadpv2(2));
cont1.dv(n) = cell2mat(tempreaddv(2));
cont1.sp(n) = cell2mat(tempreadsp(2));
cont1.mv(n) = cell2mat(tempreadmv(2));
end

```

Figure 1: MUPP OPC program created for retrieving data from the PP OPC server.

3.1 Cascade Vs. Single-loop Testing

Two functions needed to be tested with respect to the cascade controller developed. First, the intended function of any cascade controller; to increase the speed of response of a controlled system. Secondly, and this is a somewhat novel concept, to allow the outer-loop controller to command a real value input into the system and have the inner-loop work on achieving this real value.

To test the first function, appropriate controller parameters would first have to be established, which was achieved by implementing a practical stability margin tuning method. That is, first integrator time was increased to a level where it became negligible ($\tau_i = 100$ minutes), then controller gain (K_c) was increased until a large amount of oscillation was observed, indicating the system was approaching the verge of stability. This was proceeded by a reduction in controller gain of approximately 90% to ensure a non-aggressive controller. At this stage, integrator time was decreased until an adequate response was obtained. This same process was followed for tuning the outer- and inner-loop PI controllers. With tuning parameters set, controller performance testing was carried out using setpoint steps from 50-60% for both cascade and single-loop control schemes.

The second function was tested with the same controller parameters found in the previous test. This time setpoint step testing was also carried out; however, two tests were performed in this case, and in both cases, only small steps were implemented. The first step consisted of an SP change from 50 to 52%, and the second, an SP change from 52 to 50%. In these tests, the response of the controller's process variable was ignored. Instead, the focus was put on how close the outer-loop controller's output was to the manipulated variable injected into the physical system.

3.2 Feedforward Control Testing

The first step of feedforward controller testing was determining the transfer function (TF) of the feedback controller (g_{ff}), given by:

$$g_{ff} = \frac{g_d}{g}$$

Where: g_d = disturbance TF

g = process TF

As a result, models of the disturbance and process needed to be obtained. To do this, the BMT would be controlled using the cascade controller developed in the previous step. Under control, the BMT was left until steady state liquid level was reached, at which point the controller's manipulated variable would be held at its steady-state value. Thus, the system was no longer under control but remained in its steady state. The first test could be performed from this state to obtain a process response to a change in the DV, achieved with a 2% positive step to the disturbance variable; the cyclone recycle pump (CRP) speed. Given the capacity nature of the systems being studied, the system could not be left to reach some new steady state as this would never be achieved; instead, the system was observed for 500 seconds. After the first step test, the system was returned to its original steady state. For the second step test, performed to obtain a process response to a change in the MV, a 2% positive step was introduced via the systems

manipulated variable, the feed pump (FP) speed. As in the previous test, the system was observed for 500 seconds.

Once imported into a separate MATLAB program, analysis of the two tests was performed to determine the TF of the feedforward controller. The investigation called for the normalisation of collected data, the process of which was as follows:

1. Concerning the flow rate data collected for FT-148 (*Flow transmitter immediately following the FP*) and FT-348 (*Flow transmitter immediately following the CRP*), the mean of values collected once the flow had settled at steady state were obtained.
2. Using the mean collected, both flows and their corresponding liquid level responses were normalized to a response to a unity step in each variable, i.e.:

$$\widehat{pv} = \frac{pv}{\overline{mv}}$$

$$\widehat{mv} = \frac{mv}{\overline{mv}}$$

Where: $\widehat{pv}$ = the normalised PV vector
 $\widehat{mv}$ = the normalised MV vector
 $\overline{mv}$ = the mean of the MV

3. Using the normalized data, the required feedforward controller could be approximated, i.e.:

$$g_{ff} = \frac{g_d}{g}$$

Once the feedforward controller was obtained, it was implemented in EPKS as part of a Cascade Feedback Augmented Feedforward (CFBFF) Controller. At this stage, the CFBFF controller's disturbance rejection performance was compared to a Cascade controller in the same way Cascade controllers were compared to single-loop controllers in the previous step. In this case, step testing focused on the comparison of each controller's response to an identical change in disturbance flow.

3.3 OPC Control Testing

The final control scheme that required testing was the OPC control performance. After careful consideration, it was concluded that the most effective method for controller testing would be to set up identical controllers in both EPKS and via OPC in MATLAB and evaluate the differences in performance for SP step testing. The expectation was that some performance degradation would be observed in the shift from the pure EPKS configuration to the hybrid EPKS OPC design.

To fully appreciate the requirements of testing to be performed, it is worthwhile first outlining the intricacies of the designed controller. Figure 2 illustrates the simplified controller in question; here, the controller's contents are surrounded by a green box, and the physical system, by a red box. While operated exclusively by EPKS, all control elements are handled by the Experion system, with the information flowing from the SP through the intersection of the PV and becoming the error, which then

flows over the magenta line (Figure 2, b) through g_{c1} (The outer-loop PID). From the output of g_{c1} , the control signal then intersects PV2, becoming the inner-loop or cascaded error (Figure 2, c), before travelling through g_{c2} (The inner-loop PID), which outputs the MV to the physical system (Figure 2, d). While operating in the hybrid EPKS, OPC scheme, the only differing element of the controller is the outer-loop route path and g_{c1} , which in this case is routed through the blue line, out of EPKS via OPC, and into a third-party OPC compliant software (for the purpose of this testing, MATLAB). Once in the MATLAB, the information gathered via OPC can be used to form the output of any desired controller before being outputted, again via OPC, back to EPKS, where it is fed into the following control element g_{c2} . While in operation, any of the control elements can be routed to the control module output, thus, becoming the system MV, and allowing the controller to be easily configured as a single-loop or cascade system control scheme.

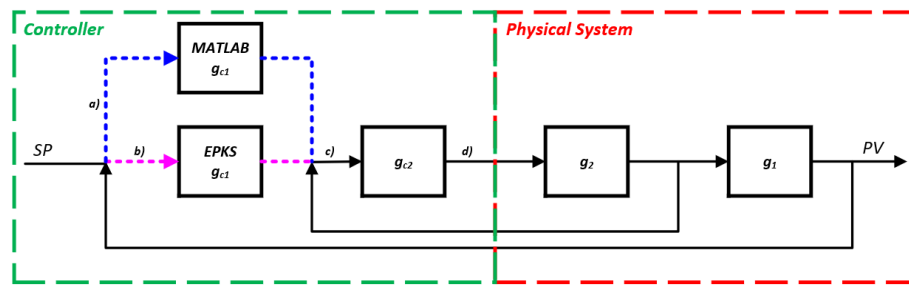


Figure 2: Block diagram of the designed controller and simplified physical system: Control flow can be configured from EPKS HMI such that MATLAB achieves outer-loop cascade control via OPC (a) OR through EPKS CM (b). Outer-loop controller output is then fed into the inner-loop controller (c) housed in the EPKS CM, the output of which is fed into the physical system (d). NOTE: Outer-loop controllers can also be fed directly into the physical system, thus becoming the controller under a single-loop control scheme.

Testing OPC performance using MATLAB was achieved by leveraging the program developed for testing the previous control schemes; however, the program did have to be adapted to incorporate a PI controller (see Figure 3). The PI was implemented as a discretised PI controller in the velocity form, i.e.:

$$mv_k = mv_{k-1} + \Delta mv$$

$$\text{where: } \Delta mv = K_c \left[\left(1 + \frac{dt}{\tau_i} \right) \varepsilon_k - \varepsilon_{k-1} \right]$$

For suitable testing, both EPKS and hybrid EPKS OPC schemes were configured with the same controller parameters ($K_c = 10, \tau_i = 1$ minute). Similarly, the loop time of the MATLAB controller was configured to be the same as in EPKS, with a Δt of 500 milliseconds. With the methodology of controller testing defined, SP step testing was carried out on both systems for steps from 50 to 55 percent and 55 to 50 percent to compare performance.

```

% MUPP -- OPC Control Testing & Data Collection
clear, close all

% Controller parameters
K=10; ti=60; dt = 0.5;

%%      Initialization of Server Comms
% Connect to MUPP OPC Server
server = opcda('ppserver1', 'HWHsc.OPCServer');
connect(server);

% Establish OPC Group i.e. EPKS Control Module to retrieve data from
grp = addgroup(server, 'FP_REF_141-R1A');

% Define variables to be read
% Reads
fp141_read = additem(grp, 'FP_REF_141-R1A.FP_141.PV', 'double');
pv1_read = additem(grp, 'FP_REF_141-R1A.PV1.PV', 'double');
pv2_read = additem(grp, 'FP_REF_141-R1A.PV2.PV', 'double');
dv_read = additem(grp, 'FP_REF_141-R1A.DV.PV', 'double');
sp_read = additem(grp, 'FP_REF_141-R1A.SP.PV', 'double');
mv_read = additem(grp, 'FP_REF_141-R1A.MV_OuterLoop.PV', 'double');
opcEnable = additem(grp, 'FP_REF_141-R1A.OPCenable.PVFL', 'double');

% Writes
fp141_write = additem(grp, 'FP_REF_141-R1A.FP141_OPC.PV', 'double');

%%      Initialization of read variables
%      Perform first read >> convert read to cell
tempreadfp = struct2cell(read(fp141_read));
tempreadpv1 = struct2cell(read(pv1_read));
tempreadpv2 = struct2cell(read(pv2_read));
tempreaddv = struct2cell(read(dv_read));
tempreadsp = struct2cell(read(sp_read));
tempreadmv = struct2cell(read(mv_read));
tempOpcState = struct2cell(read(opcEnable));

%      Take second entry from initial read (This is the value of the
%      variable) and convert to double.
cont1.fp141(1) = cell2mat(tempreadfp(2)); % \
cont1.pv1(1) = cell2mat(tempreadpv1(2)); % | Save variables as structure
cont1.pv2(1) = cell2mat(tempreadpv2(2)); % | so they can be easily
cont1.dv(1) = cell2mat(tempreaddv(2)); % | saved and retrieved by
cont1.sp(1) = cell2mat(tempreadsp(2)); % | other programs.
cont1.mv(1) = cell2mat(tempreadmv(2)); % |
cont1.OPC(1) = cell2mat(tempOpcState(2)); % /
cont1.mvDes(1) = 0; % <-- Desired MV to be written to OPC server
cont1.BMTerr(1) = 0; % <-- Error

%Initialize loop and time for object send
n = 1;
cont1.t(1) = 0;

%Initialize loop to run indefinitely
while true
    tempOpcState = struct2cell(read(opcEnable));
    cont1.OPC(n+1) = cell2mat(tempOpcState(2));

    % UNCOMMENT FOR REAL TIME PLOTTING
    % pause(dt)
    % subplot(2,1,1)
    % plot(cont1.t, cont1.pv1, 'b', cont1.t, cont1.sp, 'r')
    % grid on
    % subplot(2,1,2)
    % plot(cont1.t, cont1.mvDes, 'b', cont1.t, cont1.mv, 'r')
    % grid on
    % hold on

    % Progress count and time
    n = n+1; cont1.t(n) = n*dt;

    %      Perform next read >> convert read to cell
    tempreadfp = struct2cell(read(fp141_read));
    tempreadpv1 = struct2cell(read(pv1_read));
    tempreadpv2 = struct2cell(read(pv2_read));

```

```

tempreaddv = struct2cell(read(dv_read));
tempreadsp = struct2cell(read(sp_read));
tempreadmv = struct2cell(read(mv_read));

%           Take second entry from initial read (This is the value of
%           the variable) and convert to double.
cont1.fp141(n) = cell2mat(tempreadfp(2));
cont1.pv1(n) = cell2mat(tempreadpv1(2));
cont1.pv2(n) = cell2mat(tempreadpv2(2));
cont1.dv(n) = cell2mat(tempreaddv(2));
cont1.sp(n) = cell2mat(tempreadsp(2));
cont1.mv(n) = cell2mat(tempreadmv(2));

%% Controller
if (cont1.OPC(n) == 1)
    % ^^ If OPC enabled in EPKS

    cont1.BMTerr(n) = cont1.sp(n) - cont1.pv1(n);    % <-- Error

    tempmvDes = cont1.mvDes(n-1) + K*((1+(dt/ti))*cont1.BMTerr(n) - ...
        cont1.BMTerr(n-1));
    % ^^ Velocity form discretised PI controller

    if tempmvDes > 105          % <-- if MV > maxMV make MV = maxMV
        cont1.mvDes(n) = 105;
    elseif tempmvDes < -5      % <-- if MV < minMV make MV = minMV
        cont1.mvDes(n) = -5;
    else                       % <-- else make MV = temp from line 95
        cont1.mvDes(n) = tempmvDes;
    end
else

    % \/\ If OPC disabled in EPKS and ML make error zero and make
    %      MATLAB desired MV track EPKS outer loop MV.
    cont1.BMTerr(n) = 0;
    cont1.mvDes(n) = cont1.mv(n);
end

write(fp141_write, cont1.mvDes(n)); % <-- Write MV to EPKS OPC Server
end

```

Figure 3: MUPP OPC program created for testing MUPP control from MATLAB via OPC.

4.0 Results

Testing of the cascade control system was first concerned with comparing the relative performance of the single-loop and cascaded systems; this was done with a single-step test from 50 to 60 percent maximum tank height, shown in Figure 4. From this testing, it can be observed that the difference in controller performance is negligible. Figures 5 and 6, however, show excellent inner loop PV tracking for the cascade systems.

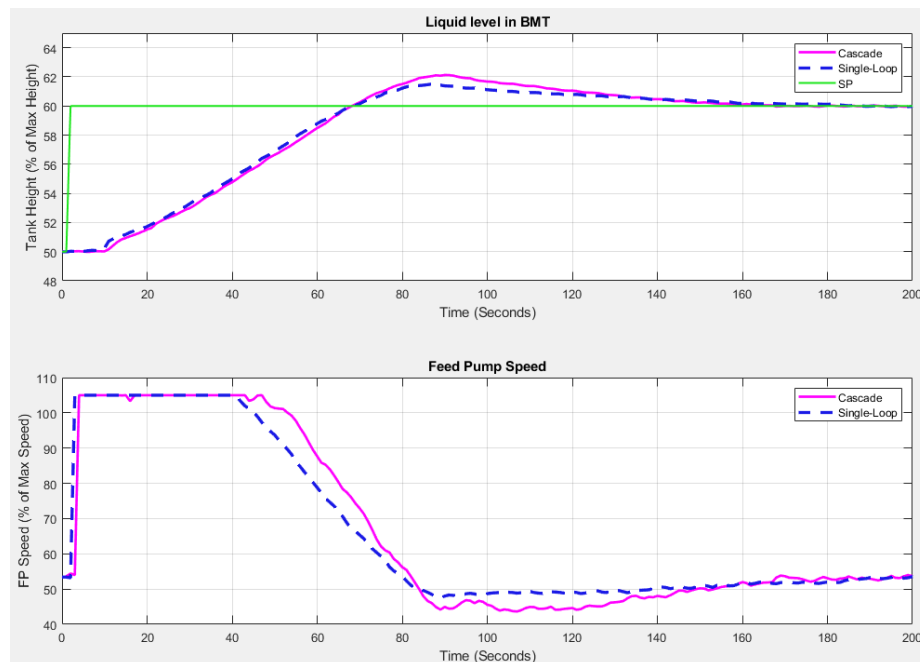


Figure 4: A comparison of Cascade versus single-loop feedback control, plots show BMT liquid level (*top*) and controlled feed pump speed (*bottom*). The process dynamics observed are in response to a change in setpoint from 50 to 60% maximum liquid level.

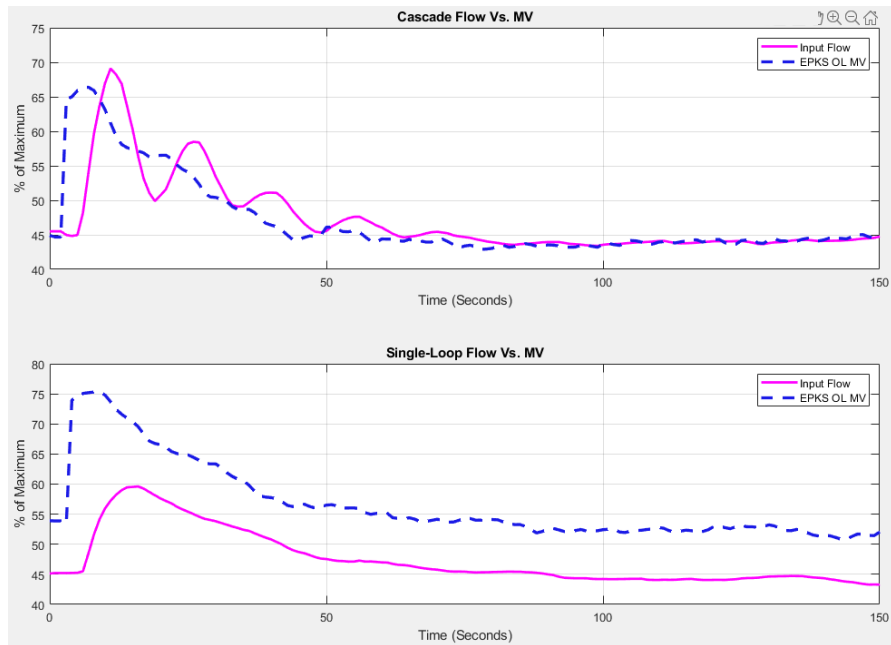


Figure 5: A comparison of Cascade (*top*) versus single-loop feedback (*bottom*) control plots show inflow into BMT compared to the outer-loop manipulated variable. The process dynamics observed are in response to a change in setpoint from 50 to 52% maximum liquid level.

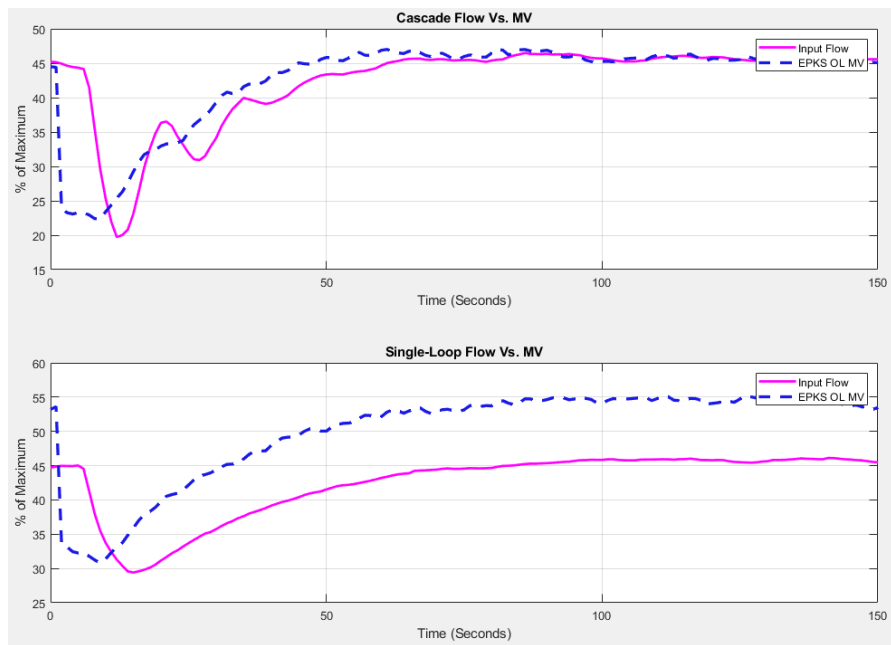


Figure 6: A comparison of Cascade (*top*) versus single-loop feedback (*bottom*) control plots show inflow into BMT compared to the outer-loop manipulated variable. The process dynamics observed are in response to a change in setpoint from 52 to 50% maximum liquid level.

Normalizing the data collected for the step testing on disturbance (CRP) and manipulated (FP) variables flowing into the ball mill tank gives rise to the data conveyed in Figure 7. Analysing the data obtained using the methodology outlined in section 5.2 results in a feedforward controller of the form:

$$G_{FF} \approx -1.4$$

Implementing this as a CFBFF controller leads to the step test results shown in Figure 8, which shows a comparison in disturbance rejection performance between the cascade feedback augmented feedforward and pure cascade feedback controllers. The comparison shows a notable distinction in disturbance rejection performance between the two control schemes, with the CFBFF controller fairing much better.

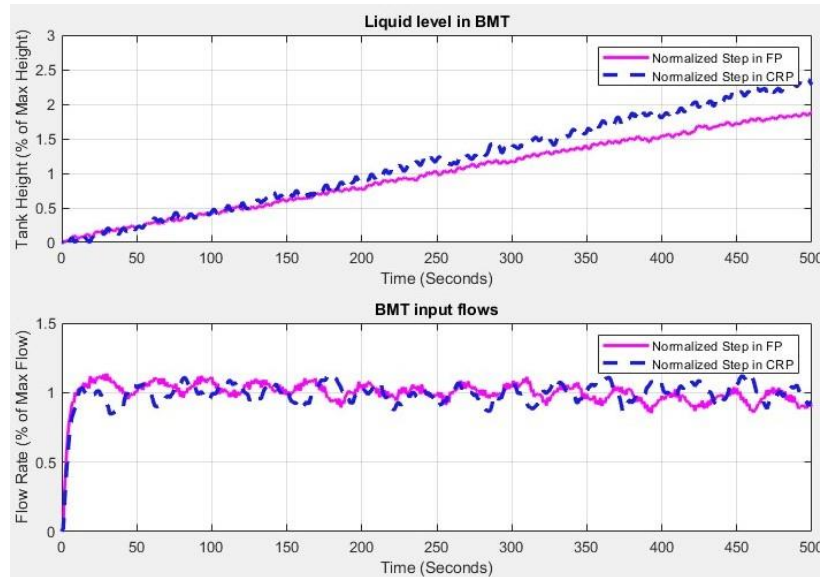


Figure 7: Response to step testing of FP and CRP, and the corresponding liquid levels (*top*) and flow rates into BMT (*bottom*) observed as a result. All step testing was implemented as a 2% increase in pump speed, then normalized based on the associated flow rate.

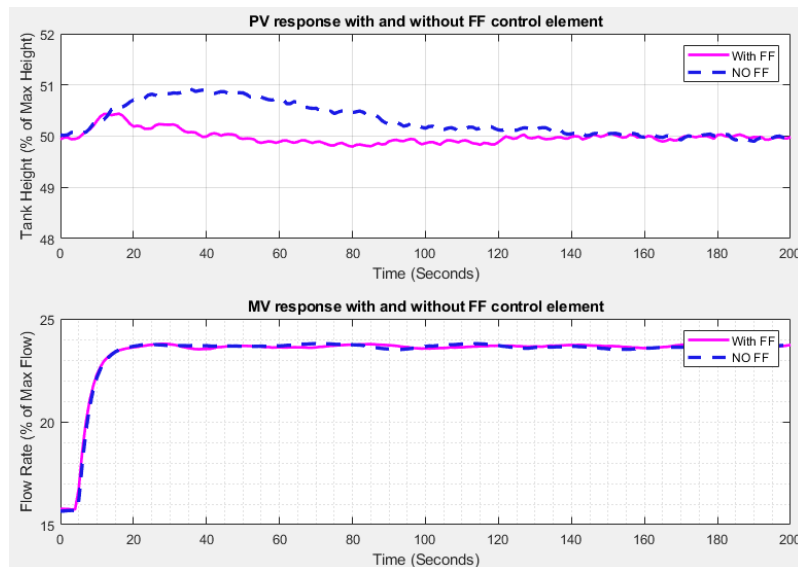


Figure 8: A comparison of Cascade Feedback Augmented Feedforward versus, Cascade-only feedback control disturbance rejection qualities. The process dynamics observed are a response to a 20% step in CRP speed from 30-50%, resulting in a ~8% increase in flow rate % from ~16-24% disturbance flow rate. (*top*) shows liquid level as a percentage of maximum height in the BMT, (*bottom*) shows disturbance variable (FT-347) for each controller during testing.

Finally, a comparison of cascade controllers with outer-loop PI controllers housed in EPKS and MATLAB via OPC is illustrated in Figure 9 and Figure 10, which show comparative dynamics for setpoint changes of 50 to 55 and 55 to 50, respectively. Citing the two figures, it is worth reiterating that in both cases, controllers are operated with identical parameters ($K_c = 10, \tau_i = 1$ minute) and under an identical sample rate of 2 Hertz.

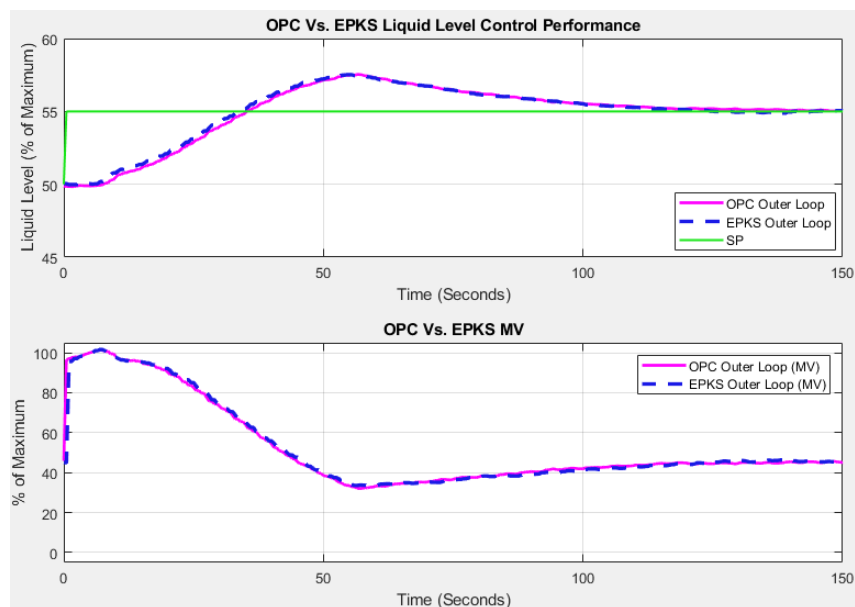


Figure 9: Performance of cascade controllers to a 5% increase in SP from 50 to 55%. The comparison shows the outer loop housed in the EPKS control module versus the outer loop housed in the MATLAB program and communicated via OPC to the EPKS control module. In both cases, configuration parameters are identical, (*top*) shows the liquid level in BMT, and (*bottom*) shows pump speed.

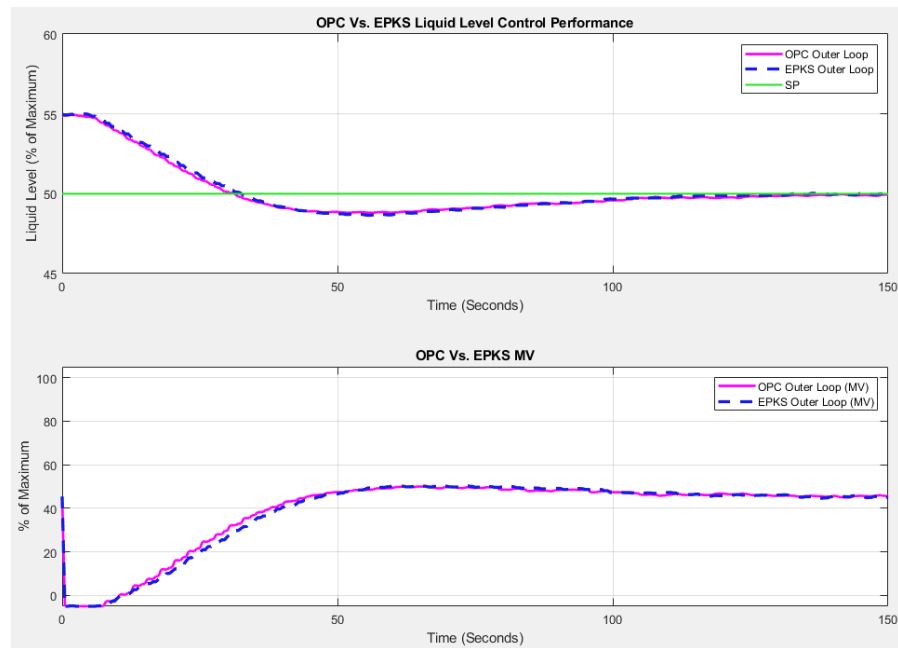


Figure 10: Performance of cascade controllers to a 5% increase in SP from 55 to 50%. The comparison shows the outer loop housed in the EPKS control module versus the outer loop housed in the MATLAB program and communicated via OPC to the EPKS control module. In both cases, configuration parameters are identical, (*top*) shows the liquid level in BMT, and (*bottom*) shows pump speed.

5.0 Discussion

5.1 Cascade and Single-Loop Testing

The performance gained through the addition of a cascaded controller was insignificant (*Figure 4*); this is most likely a result of the MUPP location in which the controller is located, as opposed to an incorrectly configured cascade control scheme. Observing *Figure 4* and *Figure 5*, we can see that the configured cascade controller performs as intended. Here we see a clear distinction between the cascaded and single-loop schemes; this is with respect to the systems manipulated variable tracking outer-loop controller output. There are more suitable locations within the MUPP to include a cascade control structure. Still, with respect to controller testing, the three figures show the controller operating expectedly.

Circling back to *Figure 4* and *Figure 5*, the accuracy of MV tracking due to the inner-loop controller lends to the somewhat novel control implementation mentioned previously. If the flow transmitters contained in the MUPP were to be calibrated appropriately to measure a known quantity, say $\frac{L}{min}$, then controllers could be configured in a third-party program to work with parameters with dimension, thus, allowing the use of a multitude of advanced control strategies, otherwise impossible within EPKS. This task would have previously been implemented within the third-party program, which would have taken the sensor reading from EPKS, applied some conversion for use by the controller, calculated control action, and then applied the reverse conversion before writing the value to the EPKS server. For obvious reasons, removing the need for conversion is a desirable control system trait.

5.2 Feedforward Testing

The testing of the feedforward controller showed a fully functional CFBFF controller with a clear distinction in performance between the feedback augmented and pure feedback systems. The system with a feedforward component observed an improvement of ~60% in terms of reduced setpoint deflection and a >80% improvement in overall disturbance rejection.

5.3 OPC Performance

As stated in the preceding sections, there was the expectation that the shift in control of the outer loop from EPKS to a third-party application would result in some degradation in control performance. The result of testing, however, showed no perceivable loss of performance. In fact, observing *Figure 9* and *Figure 10*, there is a negligible difference between the two controllers with identical controller parameters. Some minor variations can be seen throughout testing; however, these discrepancies are just as likely due to slight variations in the process conditions between the two tests as they are a result of differences in the controller. As a result, it can be concluded that OPC performance is more than adequate in terms of justifiable MUPP control schemes.

6.0 Conclusion

Following testing, it can be concluded that the controller and all its constituent components operated as intended. The testing of functionality included the following control schemes:

- Single- and Cascaded-loop feedback controllers.
- Cascade Feedback augmented Feedforward controllers,
- Hybrid control in MATLAB via OPC.

As testing showed the correct function of all schemes, the designed controller can now be implemented for all manipulated variables in the MUPP.